

RELATÓRIO DE ALTERAÇÕES

Gestor Financeiro - Web

Análise, Correções e Melhorias

Data de emissão: 13/06/2026

Total de alterações: 13

4 críticas · 7 médias · 2 leves

Sumário das Alterações

ID	Severidade	Título	Arquivo Principal
S01	Crítico	Token JWT por requisição (HorseApiService stateless)	horseApi.ts
S02	Crítico	JWT Secret sem fallback hardcoded	config.ts
S03	Crítico	CORS restrito por variável de ambiente	index.ts
S04	Médio	Variáveis não utilizadas removidas	horseApi.ts
S05	Médio	Validação de entrada nas rotas do BFF	clientes.ts
S06	Médio	Função utilitária toSnakeCase (eliminação de repetição)	horseApi.ts
S07	Médio	Endpoint de Dashboard com dados reais	horseApi.ts
S08	Crítico	HttpOnly Cookie para autenticação (removido localStorage)	auth.ts
S09	Médio	DataTable com @tanstack/react-table	DataTable.tsx
S10	Leve	Lazy Loading de rotas + Spinner componente	App.tsx
S11	Leve	Interceptor 401 com notificação ao usuário	api.ts
S12	Médio	Tratamento de erro tipado (redução de any)	*.ts
S13	Médio	Testes automatizados (server + client)	horseApi.test.ts

Token JWT por requisição (HorseApiService stateless)

Arquivos modificados:

- server/src/services/horseApi.ts
- server/src/middleware/auth.ts
- server/src/routes/*.ts

Descrição:

O HorseApiService era um singleton que armazenava o token JWT como estado interno (this.token). Em um servidor Node.js com múltiplos usuários simultâneos, o primeiro login definia o token e requisições subsequentes de outros usuários sobrescreviam esse token, causando vazamento de dados entre contas.

Solução: Removido o estado mutável da classe. O token agora é extraído do cabeçalho Authorization ou do cookie httpOnly pelo middleware de autenticação e passado como parâmetro para cada método do serviço.

Cada requisição carrega seu próprio token de forma isolada.

Antes:

```
class HorseApiService {
  private token: string | null = null;

  async listarClientes() {
    const token = await this.getToken();
    ...
  }
}

export const horseApi = new HorseApiService();
```

Depois:

```
class HorseApiService {
  // Sem estado! Token é parâmetro.
  async listarClientes(token: string) {
    ...this.getAuthHeaders(token)
  }
}

export const horseApi = new HorseApiService();
```

Impacto:

IMPACTO: ALTO. Corrige vulnerabilidade crítica de segurança onde usuários poderiam acessar dados de outros usuários. O BFF agora é seguro para ambientes multi-usuário.

JWT Secret sem fallback hardcoded

Arquivos modificados:

- server/src/config.ts
- server/.env.example

Descrição:

O arquivo config.ts tinha um fallback hardcoded para o JWT_SECRET:

```
jwtSecret: process.env.HORSE_JWT_SECRET || 'c7f9a1b2-48d3-4e6a-9d8a-2f1e6c4a9b7d'
```

Se a variável de ambiente não estivesse configurada em produção, o secret era público e conhecido.

Solução: Criada função requiredEnv() que lança erro em startup se a variável não estiver definida. O .env.example foi atualizado com um valor placeholder.

Antes:

```
jwtSecret: process.env.HORSE_JWT_SECRET || 'c7f9a1b2-48d3-4e6a-9d8a-2f1e6c4a9b7d'
```

Depois:

```
function requiredEnv(name: string): string {  
  const value = process.env[name];  
  if (!value) throw new Error(...);  
  return value;  
}
```

```
jwtSecret: requiredEnv('HORSE_JWT_SECRET')
```

Impacto:

IMPACTO: ALTO. Impede que um secret padrão conhecido seja usado em produção. O servidor falha ao iniciar se a configuração não for explícita.

CORS restrito por variável de ambiente

Arquivos modificados:

- server/src/index.ts
- server/src/config.ts
- server/.env.example

Descrição:

CORS estava configurado com origin: '*', permitindo qualquer site fazer requisições ao BFF.

Solução: Adicionada config.corsOrigin lida do .env, com suporte a múltiplas origens (separadas por vírgula). Também adicionado credentials: true para suporte a cookies httpOnly.

Antes:

```
app.use(cors({ origin: '*' }));
```

Depois:

```
const corsOrigins = config.corsOrigin.split(',').map((s) => s.trim());  
app.use(cors({ origin: corsOrigins, credentials: true }));
```

Impacto:

IMPACTO: ALTO. Em produção, apenas origens explicitamente configuradas poderão acessar a API.

Variáveis não utilizadas removidas

Arquivos modificados:

- server/src/services/horseApi.ts

Descrição:

Em todos os métodos do HorseApiService, a variável `const token = await this.getToken()` era declarada mas nunca utilizada — os headers eram obtidos via `this.getAuthHeaders()`. Isso gerava warnings e poluía o código.

Solução: Removidas todas as declarações de variável não utilizadas, já que o token agora é parâmetro do método.

Impacto:

IMPACTO: BAIXO. Melhora a qualidade do código e elimina warnings do compilador.

Validação de entrada nas rotas do BFF

Arquivos modificados:

- server/src/routes/clientes.ts
- server/src/routes/fornecedores.ts
- server/src/routes/categorias.ts
- server/src/routes/contasPagar.ts
- server/src/routes/contasReceber.ts

Descrição:

As rotas não validavam os dados recebidos antes de enviá-los à API Horse. Qualquer payload malformato chegava ao backend legado sem filtro.

Solução: Adicionada validação básica de campos obrigatórios (nome, descrição) com mensagens de erro claras em português antes de chamar o serviço.

Antes:

```
router.post('/', authMiddleware, async (req, res) => {  
  const result = await horseApi.salvarClientes(clientes);  
  res.json(result);  
});
```

Depois:

```
router.post('/', authMiddleware, async (req, res) => {  
  if (!clientes.length || !clientes[0].nome) {  
    res.status(400).json({ error: 'Nome do cliente é obrigatório' });  
    return;  
  }  
  const result = await horseApi.salvarClientes(req.token!, clientes);  
  res.json(result);  
});
```

Impacto:

IMPACTO: MÉDIO. Evita que dados inválidos ou incompletos cheguem à API Horse, melhorando a integridade.

Função utilitária toSnakeCase (eliminação de repetição)

Arquivos modificados:

- server/src/services/horseApi.ts

Descrição:

A conversão manual de camelCase para snake_case era feita de forma repetitiva em salvarContasPagar e salvarContasReceber, com mapeamento campo a campo propenso a erros.

Solução: Criadas funções genéricas toSnakeCase() e mapKeysToSnake() que convertem automaticamente todas as chaves de um objeto, eliminando a duplicação.

Antes:

```
const mappedContas = contas.map((conta) => {  
  const { dataPagamento, ...rest } = conta as any;  
  return {  
    codigo: rest.codigo || rest.id,  
    descricao: rest.descricao,  
    data_vencimento: rest.dataVencimento,  
    fornecedor_id: rest.fornecedorId,  
    id_categoria: rest.idCategoria,  
  };  
});
```

Depois:

```
const mappedContas = contas.map((c) => {  
  const { dataPagamento, dataRecebimento, ...rest } = c as unknown as Record<string, unknown>;  
  return mapKeysToSnake(rest);  
});
```

Impacto:

IMPACTO: MÉDIO. Reduz duplicação de código e elimina risco de esquecer de mapear um campo ao adicionar novas propriedades.

Endpoint de Dashboard com dados reais

Arquivos modificados:

- server/src/services/horseApi.ts
- server/src/routes/dashboard.ts
- server/src/index.ts
- server/src/types.ts
- client/src/pages/Dashboard.tsx
- client/src/components/charts/DashboardChart.tsx

Descrição:

O Dashboard anteriormente fazia requisições paralelas para /contas-pagar e /contas-receber e calculava os totais no frontend. O gráfico usava dados mockados com valores zero.

Solução: Criado endpoint /api/dashboard que agrega os dados no servidor, incluindo:

- Totais a receber/pagar, recebido/pago
- Contas pendentes e atrasadas
- Receitas e despesas dos últimos 6 meses para o gráfico

Impacto:

IMPACTO: MÉDIO. Dashboard agora reflete dados reais. Reduz o número de requisições do cliente (1 em vez de 2). O gráfico de barras agora mostra a evolução mensal real.

HttpOnly Cookie para autenticação (removido localStorage)

Arquivos modificados:

- server/src/routes/auth.ts
- server/src/middleware/auth.ts
- server/src/index.ts
- server/package.json
- client/src/lib/api.ts
- client/src/context/AuthContext.tsx
- client/src/types/index.ts
- client/src/pages/Login.tsx

Descrição:

O token JWT era armazenado no localStorage do navegador, vulnerável a ataques XSS. Qualquer script injetado poderia roubar o token.

Solução:

- Login agora define um cookie httpOnly (inacessível via JavaScript) com o token
- O middleware de autenticação lê o token do cookie como fallback
- As requisições Axios usam withCredentials: true
- O tipo User não inclui mais token
- O interceptor 401 dispara um evento customizado que redireciona ao login com mensagem de sessão expirada
- Adicionado cookie-parser ao servidor

Impacto:

IMPACTO: ALTO. Elimina a vulnerabilidade de roubo de token via XSS. Sessão gerenciada pelo servidor.

DataTable com @tanstack/react-table

Arquivos modificados:

- client/src/components/ui/DataTable.tsx
- client/src/pages/Cientes.tsx
- client/src/pages/Fornecedores.tsx
- client/src/pages/Categorias.tsx

Descrição:

O componente DataTable.tsx existia mas não utilizava o @tanstack/react-table (que já estava instalado). Além disso, não era usado por nenhuma página.

Solução: Reescrito o DataTable usando useReactTable, getCoreRowModel e getSortedRowModel. Atualizadas as páginas de Cientes, Fornecedores e Categorias para utilizar o componente. Adicionado suporte a ordenação por coluna com indicadores visuais (setas).

Impacto:

IMPACTO: MÉDIO. Código mais limpo e padronizado. A lib @tanstack/react-table agora é efetivamente utilizada, eliminando a dependency orphan. Usuários podem ordenar colunas clicando nos cabeçalhos.

Lazy Loading de rotas + Spinner componente

Arquivos modificados:

- client/src/App.tsx
- client/src/components/ui/Spinner.tsx

Descrição:

Todas as páginas eram importadas estaticamente no App.tsx, aumentando o bundle inicial.

Solução:

- Usado React.lazy() + Suspense para carregamento sob demanda
- Criado componente Spinner reutilizável (com variantes de tamanho e modo fullPage)
- Substituídas as repetições manuais de div.animate-spin nas páginas (Clientes, Fornecedores)

Impacto:

IMPACTO: BAIXO. Reduz o tamanho do bundle inicial. O Spinner centralizado facilita manutenção.

Interceptor 401 com notificação ao usuário

Arquivos modificados:

- client/src/lib/api.ts
- client/src/context/AuthContext.tsx
- client/src/pages/Login.tsx

Descrição:

O interceptor 401 redirecionava silenciosamente para /login sem informar o usuário.

Solução: O interceptor agora dispara um evento customizado auth:unauthorized com a mensagem de erro. O AuthContext escuta o evento e redireciona para /login?expired=1&message=... que exibe um alerta explicando que a sessão expirou.

Impacto:

IMPACTO: BAIXO. Melhora a experiência do usuário ao informar claramente quando a sessão expira.

Tratamento de erro tipado (redução de any)

Arquivos modificados:

- server/src/routes/*.ts
- server/src/services/horseApi.ts
- server/src/types.ts
- client/src/hooks/useApi.ts
- client/src/pages/Login.tsx
- client/src/pages/Cientes.tsx
- client/src/pages/Fornecedores.tsx
- client/src/pages/Categorias.tsx

Descrição:

O código usava catch (error: any) e casts as any em vários lugares, perdendo a segurança de tipos.

Solução:

- Criada classe AppError com campo status no servidor
- Substituídos catch (error: any) por catch (error: unknown) com narrowing adequado
- Utilizadas arrow functions auto-invocadas para narrowing de axios errors
- Adicionado status à interface HorseError

Impacto:

IMPACTO: MÉDIO. Melhora a segurança de tipos e reduz bugs silenciosos em tempo de execução.

Testes automatizados (server + client)

Arquivos modificados:

- server/src/__tests__/horseApi.test.ts
- server/package.json
- client/src/__tests__/utils.test.ts
- client/package.json

Descrição:

O projeto não possuía nenhum teste automatizado.

Solução:

- Adicionado Vitest como framework de testes (server e client)
- Criados testes para o serviço HorseApi (conversão snake_case)
- Criados testes para o middleware de autenticação
- Criados testes unitários para funções utilitárias do client (formatCurrency, formatDate, isOverdue, cn)

Impacto:

IMPACTO: MÉDIO. Estabelece uma base de testes que pode ser expandida. Garante que as funções críticas funcionam conforme esperado após alterações.

Resumo Estatístico

Total de alterações	13
Críticas (segurança)	4
Médias (qualidade/arquitetura)	7
Leves (UX/manutenibilidade)	2
Arquivos server modificados	13
Arquivos client modificados	15+
Arquivos novos (testes, componentes)	6
Dependências adicionadas (server)	cookie-parser, vitest, @types/cookie-parser
Dependências adicionadas (client)	vitest